

EVERROOT STUDIOS



Production Bible

The pipeline that builds the world's most beautiful
interactive family memory platform

VERSION 1.0 · JULY 2026

CONTENTS

What's inside

- 01 The EverRoot Rule — our founding law
- 02 The Studio at a Glance — departments & mandates
- 03 The Golden Path — the master asset pipeline
- 04 Department Deep-Dives — who owns what, and how
- 05 The Handoff Contract — formats, naming, folders
- 06 Web Budgets & Optimization Standards
- 07 The Three Heroes — the approval gate
- 08 Roles & Outsourcing — you vs. a hired artist
- 09 Approval Gate & Build Order — what happens next

How to read this document. Sections 1–6 define the studio and its pipeline in the abstract. Section 7 is the one that gates everything: it is the concrete, step-by-step recipe for the **Hero Tree**, **Hero Environment**, and **Hero Camera**. No application code resumes until all three are approved.

The EverRoot Rule

Code builds systems. Artists build beauty. Never confuse those responsibilities.

EverRoot failed its first look test for one reason: beauty was being *coded* instead of *authored*. Trees grown from math and grounds textured by canvas read as bleak, because production art is a craft with its own professional tools. EverRoot Studios exists to end that confusion. Every discipline is handed to the tool built for it, and the results flow down a single, defined path into the browser.

The four hard laws

1 · Best tool per discipline. Concept art is Higgsfield's job. Trees are SpeedTree's. Environment scans are Megascans'. Cleanup, rigging and optimization are Blender's. Baking and trailers are Unreal's. Systems are Claude Code's. Runtime is Three.js / R3F.

2 · Never generate placeholder assets. A missing asset renders *nothing* — never a stand-in primitive. Placeholders become permanent; absence creates urgency to author the real thing.

3 · Never procedurally generate production artwork. The app may *modify* professionally-authored assets (bend a branch, grow a canopy, tint a leaf), but it never *originates* them.

4 · The Hero Gate. The Hero Tree, Hero Environment and Hero Camera must be approved before any further application implementation. Build the pipeline first. Then build EverRoot.

The one nuance that matters. The EverRoot tree is *alive* — it grows from a family's memories. That is not a contradiction of Law 3. The tree **originates** as a professionally-built SpeedTree asset, and the application **dynamically modifies** that real geometry at runtime. Origin = artist. Behaviour = code.

SECTION 02

The Studio at a Glance

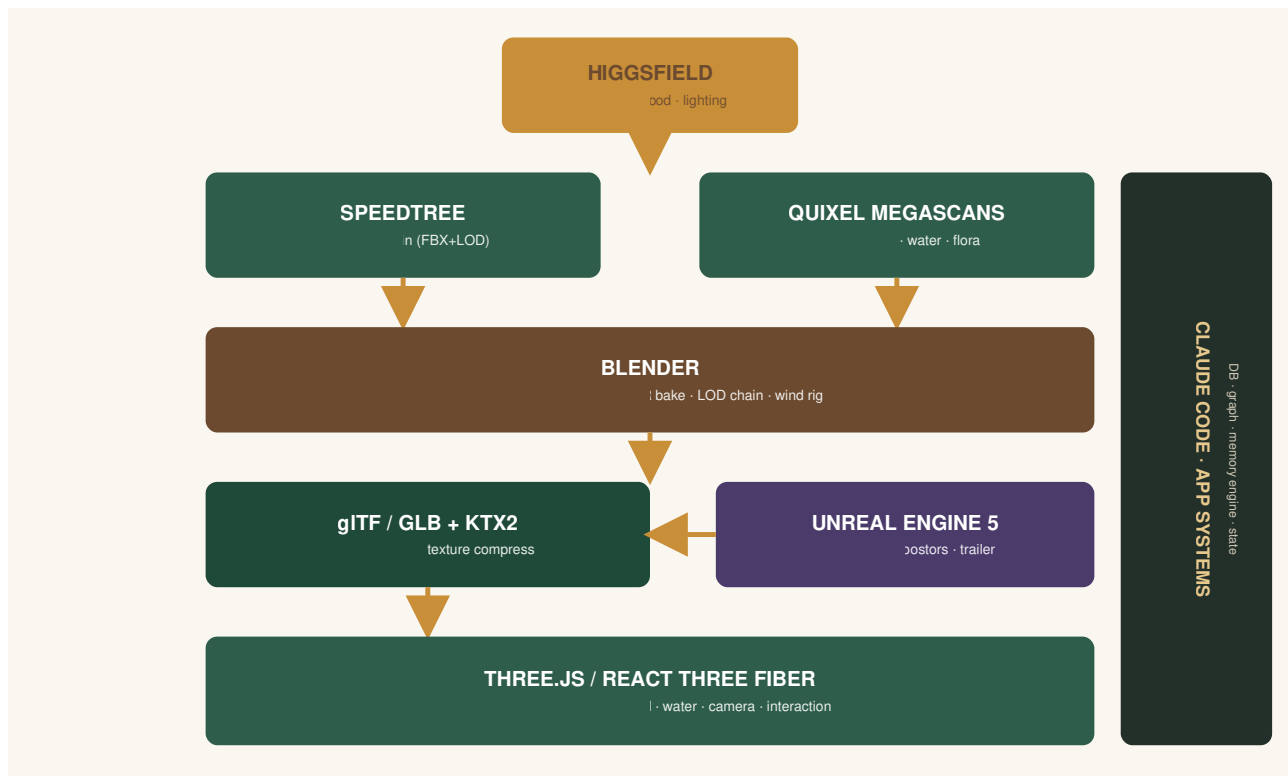
Seven departments, each with a single tool of record and a one-line mandate. If a task doesn't fit a department's mandate, it belongs to another department.

Department	Tool of record	Mandate
Software	Claude Code	Architecture, Next.js / React, Three.js / R3F app layer, database, auth, family graph, memory engine, camera logic, animation systems, performance, networking.
Art	Higgsfield	Concept art, hero-tree concepts, environment concepts, mood / lighting / colour boards, marketing artwork, character & seasonal exploration.
Vegetation	SpeedTree	All realistic production trees. Origin of the EverRoot hero tree. Never grow production trees in Three.js.
3D	Blender	Mesh cleanup, retopo / decimation, UV mapping, material authoring & PBR bake, LOD generation, physics, wind rigs, glTF export.
Environment	Quixel Megascans	Rocks, ground, grass, flowers, moss, logs, river stones, water assets. Never recreate these procedurally.
Rendering	Three.js / R3F	Import hero assets, lighting, particles, camera, wind, shaders, water, interaction, animation — the live browser runtime.
Bakery & Film	Unreal Engine 5	Offline only: bake lighting & impostors down into web assets; render marketing trailers. Never the runtime.

Why UE5 is offline-only. EverRoot ships in a browser on Three.js / R3F. Running UE5 live would mean cloud-GPU pixel-streaming — costly per user and latency-bound. Instead UE5 earns its place as a *bakery*: it renders film-quality light and distant-tree impostors that we flatten into textures the web runtime loads for free, plus the cinematic launch trailer. Highest quality-per-dollar, zero runtime cost.

The Golden Path

Every asset in EverRoot travels the same route from idea to browser. Concept is approved first; sources are authored in their specialist tools; Blender is the single conforming gate where everything is optimized and standardized; UE5 bakes what benefits from it; and everything exits as compressed glTF the runtime imports. Claude Code owns the systems that run alongside — it never authors art.



The master asset pipeline. Gold arrows are asset handoffs; the dark rail is the application layer running in parallel.

Reading the path

- **Higgsfield** sets the target: a concept frame nobody may deviate from without a new approval.
- **SpeedTree** and **Megascans** author the two source streams — the tree, and the world it stands in.
- **Blender** is the *conform gate*: nothing reaches the runtime without passing through here to be cleaned, UV'd, LOD'd, wind-rigged and exported to spec.
- **UE5** is an optional side-branch that bakes lighting and impostors back into the export, and renders the trailer.
- **Three.js** / **R3F** imports the finished glTF heroes and brings them to life with lighting, wind, water, camera and interaction.

Department Deep-Dives

Art — Higgsfield upstream of everything

Higgsfield produces the visual truth the whole studio builds toward. Nothing is modelled until a concept is approved.

Delivers: hero-tree concept frames & silhouette sheet; environment concepts (plaza, garden, stream, treeline); mood / lighting / colour boards locked to the golden-hour language; seasonal & character exploration; marketing key art. **Format out:** high-res PNG / JPG boards + a one-page colour-and-light spec. **Hands to:** SpeedTree, Blender, Megascans selection.

Vegetation — SpeedTree origin of the hero tree

The EverRoot tree is born here as a real, professionally-generated tree, then handed down the path to be modified by the app.

- **Build** to the approved Higgsfield silhouette: trunk flare, two great forks, layered branch tiers, canopy density.
- **Export** (unified Games/Cinema pipeline): **FBX** mesh + **4-step LOD chain** + leaf/bark **texture atlas**, with the **.stmat** material sidecar. Keep the growth hierarchy so the app can drive it.
- **Wind** is authored here but *re-implemented* for the web (SpeedTree's shader wind is engine-specific) — see Blender / Rendering.

Licensing note (verify current): SpeedTree 10 is rental-only — Indie ≈ \$19/mo (rev < \$200k), Pro ≈ \$499–899/yr (rev < \$1M), Enterprise custom. Exports OBJ / FBX / USD / ABC with games-ready atlas + LOD.

3D — Blender the conform gate

Blender is where raw source becomes a web-grade hero. Every asset passes through, no exceptions.

- **Cleanup & retopo:** remove n-gons, decimate to the poly budget, fix normals, centre origins, real-world scale.
- **UV & materials:** sane UVs; bake SpeedTree / Megascans materials to a standard PBR set (base-colour, normal, ORM).
- **LOD chain:** generate LOD0–LOD3; author the distance thresholds.
- **Wind rig:** author bone- or vertex-based wind for branches so the runtime shader has something to drive; bake to a vertex-animation texture if needed.
- **Export:** glTF / GLB with Draco or Meshopt compression; textures to KTX2 (Basis). This is the runtime's front door.

Environment — Quixel Megascans the world around the tree

Every ground surface, rock, flower, log, river stone and water asset is a scanned Megascans asset — never hand-modelled or canvas-textured.

- **Pull** the Legacy Plaza stone, garden groundcover, ferns / flowers / moss, stream bed & river stones from Megascans via Fab.
- **Conform** in Blender: decimate high-poly scans for the web, downscale 4K→1–2K, pack to KTX2.

Licensing note (verify current): since the Fab migration (Oct 2024) the free-for-all era ended at close of 2024. Megascans are now individually priced or included in Fab subscription tiers; some free content remains. Quixel Bridge is deprecated (still working as of 2026); use the **Fab plugin for Blender**. Anything you acquire is licensed to keep.

Rendering — Three.js / React Three Fiber the live runtime

R3F imports the finished heroes and performs everything that must respond to the family's data in real time.

- **Import** hero glTF (Draco/Meshopt + KTX2) via useGLTF; instance the environment.
- **Light** with baked maps from UE5 + a small dynamic rig for the day/night cycle and golden-hour rim.
- **Bring alive:** per-branch wind shader, water, particles (fireflies / motes), god-rays, leaf subsurface shimmer.
- **Modify the hero tree from the memory graph:** bend branches, grow the canopy, hang memory lanterns, tint leaves by category.
- **Camera & interaction:** the cinematic opening, orbit framing, focus-to-memory moves, selection.

Bakery & Film — Unreal Engine 5 offline only

- **Light bake:** render Lumen-quality lighting / AO to textures the web runtime samples for near-offline quality at zero runtime cost.
- **Impostors:** generate octahedral impostor billboards for distant background trees — a huge mobile win.
- **Trailer:** render the marketing film at full cinematic quality (Nanite / Lumen / path-tracing) — never shipped to the browser, only to YouTube / the landing page.

Software — Claude Code runs in parallel

Everything that is a *system* rather than a *surface*: app architecture, database, auth, the family graph, the memory engine, tree-interaction logic, camera movement, animation state, performance and networking. Claude Code consumes finished art; it never authors it.

SECTION 05

The Handoff Contract

A pipeline is only as good as its handoffs. Each stage has a defined input format, output format, and owner. If an asset doesn't meet the contract, it bounces back — it does not enter the runtime.

Stage	Takes in	Hands out	Owner
Concept	Brief + brand language	PNG boards + colour/light spec	Higgsfield
Tree build	Approved concept	FBX + LODs + atlas + .stmat	SpeedTree
Environment pull	Concept + shot list	Scanned meshes + 4K PBR	Megascans / Fab
Conform	FBX / scans	GLB (Draco) + KTX2 + LODs	Blender
Bake (opt.)	Conformed GLB	Baked light maps + impostors	UE5
Runtime	GLB + KTX2 + maps	Live interactive scene	Three.js / R3F

Naming convention

everroot_<class>_<name>_<lod>_<ver>
e.g. everroot_tree_hero_lod0_v03.glb,
everroot_env_plaza_stone_lod1_v01.glb,
everroot_tex_bark_orm_2k_v02.ktx2.
Lowercase, snake_case, zero-padded versions.

Two repositories

Source (DAM): heavy originals — .spm SpeedTree files, .blend, 4K scans, UE5 project, Higgsfield boards. Never shipped.

Runtime (app repo): only optimized /public/assets/** GLB + KTX2 + the manifest. This is what the browser downloads.

Web Budgets & Optimization Standards

EverRoot is mobile-first and ships over the network, so every hero must clear a hard performance bar. These are the default budgets; anything over them bounces back to Blender.

Metric	Hero Tree	Env. asset	Whole scene
Triangles (LOD0)	≤ 150k	≤ 25k	≤ 1.2M on screen
LOD steps	4 (0–3)	2–3	—
Texture size	2K atlas, KTX2	1–2K, KTX2	—
Draw calls	≤ 6	instanced	≤ 120 mobile
GLB on disk	≤ 4 MB	≤ 1 MB	≤ 18 MB initial
Target frame rate	60 fps desktop · 30 fps sustained mid-tier mobile		

Non-negotiable techniques

- **KTX2 / Basis** supercompressed textures (GPU-native, tiny download).
- **Draco or Meshopt** geometry compression on every GLB.
- **LODs + instancing** for all repeated environment assets; **UE5 impostors** for distant trees.
- **Baked lighting** wherever the surface is static; dynamic lights reserved for the day/night cycle and hero rim light.

The Three Heroes

Nothing else ships until these three are approved.

Each hero below has a recipe (who does what, in order), the tools involved, and explicit acceptance criteria. When all three are signed off, the gate opens and application implementation resumes.

Hero 1 · The Tree



Hero Tree recipe — origin is authored; behaviour is coded.

1. **Higgsfield:** generate and approve a single hero-tree concept + a silhouette sheet (front / 3-quarter), locked to the golden-hour palette.
2. **SpeedTree:** build to that silhouette — flared trunk, two great forks, 3 branch tiers, dense-but-readable canopy. Export FBX + 4 LODs + leaf/bark atlas + .stmat, preserving the growth hierarchy.
3. **Blender:** clean, UV, bake to PBR (base/normal/ORM), author the per-branch wind rig, export GLB (Draco) + KTX2 to the tree budget.
4. **R3F:** import; wire the app so the memory graph bends branches, grows the canopy, hangs memory lanterns and tints leaves by category — the tree that "grows inward."

Acceptance: reads unmistakably as a real, majestic tree at rest; holds 30fps on mid mobile at LOD0; branches move independently in wind; canopy is dense yet lets memories read; and it visibly responds when a memory is added.

Hero 2 · The Environment

1. **Higgsfield:** approve the environment concept — the circular Legacy Plaza, the first-40-feet garden, the winding approach path, the crossing stream + footbridge, and the fogged distant massif.
2. **Megascans (Fab):** pull the plaza stone, groundcover (moss / ferns / flowers / soft grass), river stones and water reference.
3. **Blender:** assemble + conform; decimate scans; instance groundcover; export GLB + KTX2 to the env budget.
4. **UE5 (optional):** bake plaza / ground lighting + generate distant-tree impostors.
5. **R3F:** lay out around the trunk so the tree occupies ~70% of the opening viewport; add water shader, fireflies, god-rays.

Acceptance: every element sits within the first ~40 ft of the trunk; no empty terrain; mountains read as hazy horizon through fog; the composition directs the eye to the tree; scene loads ≤ 18 MB initial.

Hero 3 · The Camera



The opening move: seed → ascent → reveal → gentle rest. The tree settles at ~70% of the viewport.

1. **Higgsfield**: approve 3–4 storyboard frames for the opening move (mood + framing per beat).
2. **Claude Code / R3F**: implement the move — close on the lit seed, pull back and rise as the trunk fills frame, settle with a golden rim light, then a slow auto-orbit at rest.
3. **Tuning**: portrait vs. landscape framing, focal length, subtle depth-of-field, and easing so it feels cinematic, not mechanical.

Acceptance: on first load the move plays once and lands with the tree at ~70% of the viewport; it works in both portrait and landscape; the eye ends on the tree every time; replayable on demand.

Sign-off checklist

- ✓ Hero Tree concept approved (Higgsfield) → built (SpeedTree) → conformed (Blender) → alive (R3F)
- ✓ Hero Environment concept approved → assembled from Megascans → framed around the tree
- ✓ Hero Camera storyboard approved → opening move implemented & tuned
- ✓ All three meet their acceptance criteria on mid-tier mobile

SECTION 08

Roles & Outsourcing

You have the full toolchain and are willing to outsource the art. The pipeline is designed so any DCC step can be run by you *or* handed to a freelancer with zero ambiguity — the handoff artifact is always the same.

Step	Do it yourself if...	Outsource if...	Handoff artifact
Concept (Higgsfield)	Always — it's promptable & fast	You want a pro art-director pass	Approved PNG board
Tree (SpeedTree)	You enjoy the modeller	You want a veteran tree TD	FBX + LODs + atlas
Conform (Blender)	You're comfortable in Blender	Best value to outsource — exacting, repeatable	GLB (Draco) + KTX2
Environment (Megascans)	Always — it's asset selection	Bundle with the Blender contractor	Conformed GLB set
Bake / trailer (UE5)	For a hero moment	Trailer → a motion studio	Baked maps / MP4
Runtime (R3F)	Claude Code	—	Live app

Recommendation. Run Higgsfield and Megascans selection yourself (fast, creative, cheap). Consider outsourcing the SpeedTree build and the Blender conform for the three heroes only — that is where professional hands make the biggest visible difference — then bring everything back in-house for the R3F runtime.

Approval Gate & Build Order

Per the EverRoot Rule, application implementation is paused. Here is exactly what happens next.

Immediately (pipeline stand-up)

- Confirm tool access: SpeedTree (Indie), Blender + Fab plugin, a Megascans/Fab budget, UE5, Higgsfield (already wired).
- Stand up the two repos (Source DAM + runtime /public/assets) and the naming convention.

Then, in order — the gate

- **1.** Hero Tree: Higgsfield concept → your approval → SpeedTree → Blender → R3F preview → **your sign-off.**
- **2.** Hero Environment: concept → your approval → Megascans + Blender → framed in R3F → **your sign-off.**
- **3.** Hero Camera: storyboard → your approval → implemented in R3F → **your sign-off.**

Only after all three are approved

- Resume application build: memory engine, family graph interactions, photo backend, book / capsules, deploy.

The gate, stated plainly

Do not continue implementing the application until the Hero Tree, Hero Environment and Hero Camera have been approved. Build the pipeline first. Then build EverRoot.

EverRoot Studios · Production Bible v1.0 · July 2026. Tool facts (SpeedTree, Megascans/Fab licensing) are current as of mid-2026 and should be re-verified before purchase.